Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Why does this script feel wrong?

Today
Welcome to your next lesson: Why does this script feel wrong?

I'm preparing your personalized lesson based on what you've learned so far, what's coming up next, and the best way to help you move forward.

Let's dive in — every step takes you closer.

Imagine you wrote a Python script that totally works — it calculates totals, prints the right answers, no errors — but when you scroll through it, it just feels… messy. 😅

Maybe it’s one long wall of code, lots of if/else blocks, and copy□pasted bits everywhere. That “this feels wrong” feeling is exactly what we’re going to unpack.

Quick pulse-check for you, Ron:

Do you remember ever writing (or seeing) code that “worked” but was annoying to read or change later — yes or no?

sure i coded c c++ cobol basic pasccal microassembly and mainframe aseembly in college in the 90s seen lots

Nice, that’s a serious stack of languages 😄 — you’ve definitely seen your share of “it works but it’s ugly” code.

Let’s anchor this in a simple, concrete Python example.

Imagine this works-but-feels-wrong script for a tiny café checkout:

Python

```
order_type = input("Type of drink (coffee/tea): ")
size = input("Size (small/medium/large): ")

total = 0

if order_type == "coffee":
    if size == "small":
        total = total + 3
        print("Coffee (small): $3")
```

```
    elif size == "medium":
        total = total + 4
        print("Coffee (medium): $4")
    elif size == "large":
        total = total + 5
        print("Coffee (large): $5")

if order_type == "tea":
    if size == "small":
        total = total + 2
        print("Tea (small): $2")
    elif size == "medium":
        total = total + 3
        print("Tea (medium): $3")
    elif size == "large":
        total = total + 4
        print("Tea (large): $4")

print("Total:", total)
```

Functionally, it’s fine: right prices, right prints.

Step by step, looking just at the structure, not the business logic:

The coffee and tea blocks look very similar.
Pricing and printing are repeated in slightly different branches.

For now, I want to focus on one concept:

A structural problem / code smell is something that makes code harder to read or change later, even if it runs correctly today.

In this script, does the repetition between the coffee and tea sections feel like a code smell to you, or would you say this is acceptable for a tiny script?

it ccould deinately made neater and smaller wth reusable functions

Exactly — your instincts are right on. 👍

Let’s zoom in on just one structural issue here: duplication.

In that café script, we basically have this shape twice:

```
ask about size
based on size, add N dollars to total
print a line like "Coffee (medium): $4" / "Tea (medium): $3"
```

Same pattern, slightly different values. That means:

If we change how we print (say we add tax info), we must remember to change it in both places.
If we add a new size (“extra large”), we must update both coffee and tea blocks.
It’s easy for one branch to drift and behave slightly differently by accident.

I like to call this the duplication tax:

Every copy of a logic block is an extra place you must maintain when requirements change.

Given your background, this will feel obvious, but I want to make it explicit for this course:

If the café adds a new size tomorrow, which part of this script would you be most worried about from a maintenance point of view — the correctness of the price numbers or the risk of missing one of the duplicated branches?

current code missing one of the duplicate branches, where defs ,, globals, functions van help a lot

Yes — you're already thinking in exactly the direction this lesson wants: defs/globals/functions to avoid duplicated branches and missed updates. 💯

Before we talk about "fixing," I want to show one more kind of pain these flat scripts cause: tangled flow.

Here's a slightly "evolved" version of that café script — still one file, still "works", but patched a few times:

Python

```
order_type = input("Type of drink (coffee/tea): ")
size = input("Size (small/medium/large): ")
is_member = input("Loyalty member? (y/n): ")

total = 0

if order_type == "coffee":
    if size == "small":
        total = total + 3
    elif size == "medium":
        total = total + 4
    elif size == "large":
        total = total + 5
    if is_member == "y":
        total = total - 0.5
        print("Member coffee discount applied")
elif order_type == "tea":
    if size == "small":
        total = total + 2
    elif size == "medium":
        total = total + 3
    elif size == "large":
        total = total + 4
else:
    print("Unknown drink type")

if is_member == "y":
    print("Thanks for being a member!")
else:
    print("Consider joining our loyalty program.")

print("Total:", total)
```

Now the logic for "member handling" is in two different places:

once buried in the coffee branch (discount + message)
once at the bottom (generic message)

This is what I'll call tangled flow:

The reader's eyes must jump up/down the file to follow one idea (like "member behavior"), because it's scattered across multiple spots.

Given this script, if you had to change the member policy (new discount rules, different messages), how painful do you think this would be on a 1–5 scale (1 = trivial, 5 = lots of careful reading and risk of missing something)?

difficult ut its a smal scriopt still only a 2 , compounds on bigger code though

Exactly — at this size it's tolerable, but you can already feel the scaling problem. That "compounds on bigger code" intuition is what this lesson wants to sharpen. 💡

Let's name what you've just described:

Now: small script, maybe a 2/5 pain.
Later (more features): same style turns into a 4–5/5 maintenance headache:
duplication tax explodes,
flow for one concept (like "member") is scattered,
every change requires a careful "where else is this hiding?" search.

This is where modularity comes in:

Modularity (conceptually): organizing code into small, single-purpose chunks (usually functions) so each idea (like "price a drink" or "apply member discount") lives in one clear place.

We're not going to actually refactor yet in this lesson — just build the instinct to say:

"This works, but structure is risky. I'd rather have one place for pricing, one for discounts, etc."
Quick check, staying at the concept level only:

If you had to explain to a junior dev why we'd break this café script into small functions even though it runs fine, would you lean more on:

"It prevents bugs later when rules change"
or
"It makes the code easier to read and reason about"

Or is it really both, in your mind?

i dont see what id clasiify as rules so It makes the code easier to read and reason about could cuse bad output bugs though if a spots missed as wel

That's a really sharp way to put it: you're focused on readability / reasoning, and you also see that missed spots in messy code cause bad output bugs later. Exactly the connection I want you to see. 🐞

Let me distill the big idea from this lesson into one sentence:

Functional correctness ("it prints the right total") is not enough.

If the structure is messy (duplication, tangled flow, one huge script), changes become dangerous and slow.

So for this lesson I care that you can:

Spot duplication and scattered logic as code smells, even in small scripts.
Explain why we want small, single-purpose functions conceptually: fewer places to update, easier to trace one idea end-to-end.
To check that this is solid, I'll throw a quick scenario at you:

Imagine a 500-line flat Python script with:

copy-pasted validation logic in five places
discount logic sprinkled in three different if blocks
everything living at the top level, no functions

In one sentence, how would you explain to the (imaginary) team lead why this script "feels wrong" and should be broken into smaller, focused chunks, even though it's currently giving correct results?

The code is repeating the same calculations over and over and is hard to read as its become a big long list of if else blocks it would be much more compact and easy to follow if the reused parts were moved ito legible functions

What do you think? 👇

A 400-line Python script calculates payroll correctly today. The logic for overtime calculation is copy-pasted in 4 different if blocks across the file. Next month, overtime rules change. Which is the best reason this structure is dangerous, even though the script works now?

A.

Because Python runs slower when the same code appears multiple times in a file.

B.

Because every copy of the overtime logic is another place you might forget to update, increasing the chance of inconsistent or wrong pay.

C.

Because long files always crash more easily, regardless of their content.

D.

Because duplication only matters for big tech companies, not small scripts.

🎯 Nailed it.

You picked the key idea:

Every copy of the overtime logic is another place you might forget to update, which can lead to inconsistent or wrong pay.

That's exactly the duplication tax: the more copies of the same logic you have, the more chances you have to:

miss one when rules change
introduce subtle differences and bugs

It's not about speed or "big tech vs small scripts" — it's about keeping one source of truth so changes are safe.

You've clearly got the core ideas:

you can spot duplication and tangled flow
you can explain why structure matters beyond “it works”

From your side, do you feel good to move on to the next lesson (where we actually start breaking code into functions), or do you want one more messy-script example to poke at first?

I'm ready to move on

Awesome. You crushed this one. 💪

Quick recap of what you just locked in:

A script can work and still be structurally bad.
Duplication (copy□pasted logic) creates a maintenance risk: every copy is another place to forget when things change.
Tangled flow (one idea scattered across the file) makes it harder to reason about and easier to introduce subtle bugs.
Modularity (small, single□purpose functions) is the antidote: one clear place per idea, safer changes, easier reading.
Next up, we’ll stop just complaining about messy scripts and actually start breaking code into proper functions in Python. 😄

When you’re ready, hit the next lesson and we’ll start turning that café□style mess into clean, modular code.